

Robot Dynamics Midterm 2

Prof. Marco Hutter, Victor Klemm
Teaching Assistants: Mike Zhang, Arjun Bhardwaj,
Filippo Spinelli, Pascal Roth

November 6, 2024

Duration: 1h 15min

Permitted Aids: The exam is open book, which means you can use the script, slides, exercises, etc; The use of internet (besides for licenses) is forbidden; no communication among students during the test is allowed.

1 Instructions

1. Download the ZIP file `RobotDynamics_Quiz2.2024.zip` from Moodle. Extract all contents of this file into a new folder and set MATLAB's¹ current path to this folder.
2. Run `init_workspace` in the MATLAB command line.
3. All problem files that you need to complete are located in the `problems` folder.
4. Run `evaluate_problems` to check if your functions run. This script does not test for correctness. You will get 0 points if a function does not run (e.g., for syntax errors) or does not terminate (e.g. infinite loops). Remember to rerun the setup section before testing new functions.
5. When the time is up, zip the entire folder and name it `ETHStudentID_StudentName.zip`. Your student ID can be found on the front of your ID card and has the format `xx-xxx-xxx`, please include the dashes. Submit this zip-file through Moodle under **Midterm Exam 2 Submission**. You should receive a confirmation email.
6. If the previous step did not succeed, you can email your file to `robotdynamics@leggedrobotics.com` from your ETH email address with the subject line `[RobotDynamics] ETHStudentID - StudentName`
7. **Important:**
 - (a) Implementations outside the provided templates will not be graded and receive 0 points. No external libraries are allowed, except for MATLAB's Symbolic Math Toolbox.
 - (b) Helper functions included in the `solutions/pcode` directory are used for simulating the robot. Using these functions to solve the questions (unless explicitly stated) is prohibited and will lead to 0 points.
 - (c) Helper functions in the `utils` folder should be used by students in their solutions when needed. The function body can be freely inspected, however modifications are prohibited and will lead to 0 points.

¹Online version of MATLAB at <https://matlab.mathworks.com/>

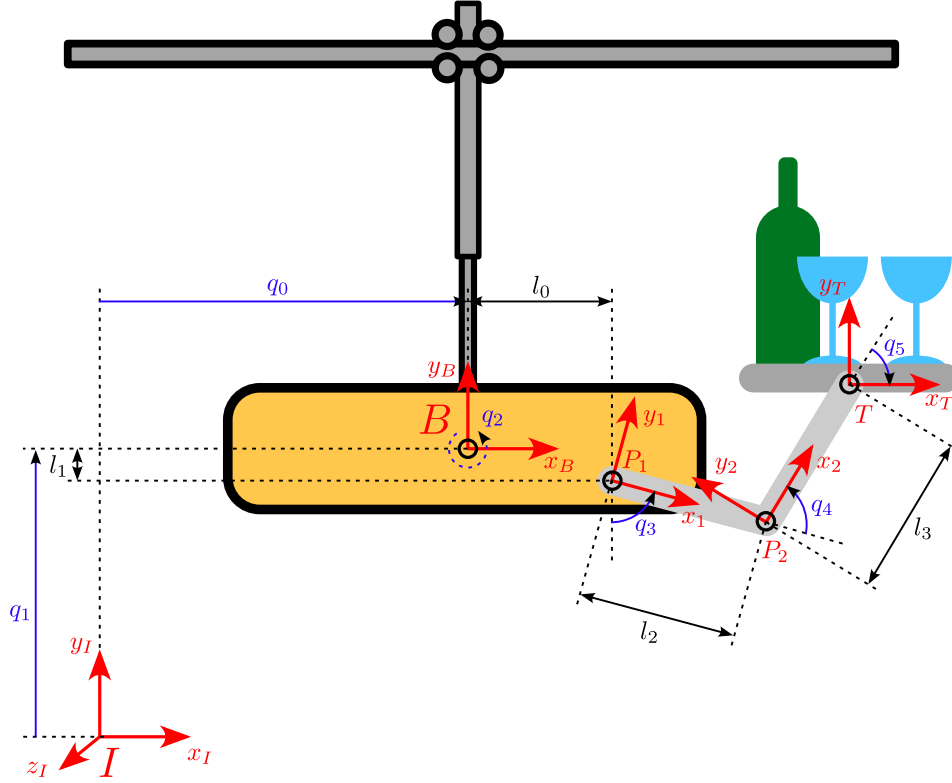


Figure 1: Diagram of the new robot waiter from the startup *Zürich Dynamics*. The robot has two prismatic joints and four rotational joints. All four rotational joints, indicated with hollow circles, rotate around the positive z axis following the right-hand rule (For clarity, the z axis is only drawn for the I frame).

2 Questions

In this quiz, you will model the dynamics for the robot waiter shown in Fig. 1. The system has 4 bodies comprising of a gantry driven base B with 3 Degrees-of-Freedom (DoF) and a 3 DoF arm attached to the base.

IMPORTANT: To simplify modeling, we assume that the links of the gantry carrying the robot base are massless and do not contribute to the equations of motion.

The reference frame attached to the base is denoted as $\{B\}$, and is modeled as two prismatic joints q_0 and q_1 , and one revolute joint q_2 . The base moves with respect to the inertial frame $\{I\}$. The reference frames attached to each link of the robot's tray carrying leg are denoted as $\{P_1\}$ and $\{P_2\}$ while the reference frame attached to the tray is denoted as $\{T\}$.

The generalized coordinates are defined as

$$\mathbf{q} = \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \\ q_4 \\ q_5 \end{bmatrix} \in \mathbb{R}^6. \quad (1)$$

Clarification: The angles q_2, q_3, q_4, q_5 are measured using the right-hand thumb rule. For the state of the scene shown in Fig. 1, q_2 would be zero, q_3 and q_4 would be positive, and q_5 would be negative.

In the following questions, all required parameters are passed to your functions in a structure called **params**. You can access it as follows:

```
1 l0 = params.l0;
2 l1 = params.l1;
3 l2 = params.l2;
4 etc..
```

In the following questions, we have already provided the kinematics (transforms, Jacobians) and controller gains (k_P , k_D) for you. Most of the variables, stored as MATLAB *cells*, may be accessed as follows:

```
1 m{k};           % mass of link k
2 k-I-s{k};       % Rotational inertia of link k in frame k
3 R-Ik{k};        % Rotation matrix from frame k to I
4 etc..
```

Remember that MATLAB indexing starts from 1, so for example $q(1) = q_0$ in our robot description.

Question 1.

6 P.

Calculate the mass matrix $M(\mathbf{q})$, nonlinear terms $\mathbf{b}(\mathbf{q}, \dot{\mathbf{q}})$ (Coriolis and centrifugal), and gravitational terms $\mathbf{g}(\mathbf{q})$ of the full robot. A helper function, `dAdt()`, is available in the `utils` folder, to compute the time derivative of a matrix.

Implement your solution in `Q1_generate_eom.m`. Kinematics equations and dynamic parameters are provided and loaded at the beginning of the file.

Question 2.

4 P.

Implement a forward dynamics simulator for the scenario where **the base link B of the robot is fixed**. We will only implement the dynamics simulation for the arm links and their corresponding generalized coordinates given by

$$\mathbf{q}_{arm} = \begin{bmatrix} q_3 \\ q_4 \\ q_5 \end{bmatrix} \in \mathbb{R}^3. \quad (2)$$

To do this, we'll compute the arm joint accelerations $\ddot{\mathbf{q}}_{arm}$ and integrates them to get $\dot{\mathbf{q}}_{arm}$ and $\mathbf{q}_{arm}$ at the next time step. You can use the the provided `M_arm`, `b_arm`, and `g_arm` for your implementation.

While we do not consider the full dynamics due to the fixed base, we must still set the value of the base generalized coordinates (q_0 , q_1 , and q_2) in our simulator. Given that the base never moves, how then can we set the base generalized coordinates for the next time step?

Implement your solution in `Q2_forward_dynamics.m`. You can check your implementation by running `simulate_Q2.m`. If `use_solution` is set to 1, executing this script will show you how the solution should look like. If `use_solution` is set to 0 then your implementation of the forward dynamics simulator will be ran.

Question 3.

4 P.

We want to be able to control the robot to follow some desired reference trajectory, leveraging the computed model of the robot's dynamics. Let's start with an impedance controller with gravity compensation.

Implement a joint-level PD controller that compensates for gravity effects and tracks desired joint positions and velocities by calculating $\boldsymbol{\tau}$, the control torque for each joint (a force for the prismatic joint).

The current joint positions $\mathbf{q}$ and joint velocities $\dot{\mathbf{q}}$, as well as the desired joint positions $\mathbf{q}^d$ and desired joint velocities $\dot{\mathbf{q}}^d$ are given to the controller as input arguments to the MATLAB function. You should obtain the gravitational terms in this question through the provided `g_fun_solution(q)`. Use the existing PD gains in the parameters (`params`).

Implement your solution in `Q3_gravity_compensation.m`. You can test your controller (or inspect the solution) by running `simulate_Q3.m`. If `use_solution` is set to 1, executing this script will show you how the solution should look like. If `use_solution` is set to 0, at each time-step the simulator will use the joint torques returned by your own implementation.

Question 4.

4 P.

In this question, we will develop a controller for tracking a trajectory of the tray. Implement an inverse dynamics controller, i.e. a controller which compensates for the full dynamics and produces the control torque for each joint τ .

Additionally, there is an external force being applied on the tray. For simplicity, we limit this external force to be only applied along the x axis of the inertial frame I . Assume that we can measure this external force perfectly. The controller should then compensate for this external force in the computed control torques τ .

The inputs to this controller are the desired tray position, desired tray orientation, desired tray linear velocity, the required compensating force, as well as the current joint position $\mathbf{q}$ and joint velocities $\dot{\mathbf{q}}$. The desired trajectory and compensating force have the following components (all expressed in the inertial frame):

- desired tray position ${}_I r_{IT}^d \in \mathbb{R}^3$
- desired tray orientation ${}_I C_{IT}^d \in \mathbb{R}^{3 \times 3}$
- desired tray velocity ${}_I v_T^d \in \mathbb{R}^3$
- the x axis force the robot must apply in order to compensate for the external force acting on the tray ${}_I F_{T_x}^d \in \mathbb{R}$

Use the mass matrix, non-linear terms and gravitational terms obtained from `M_fun_solution(q)`, `b_fun_solution(q, dq)` and `g_fun_solution(q)`. The PD gains are provided in the parameters (`params`). You can use all the helper functions in the `utils` folder. Specifically:

- `psuedoInverseMat()`, to compute the damped pseudo-inverse for any input matrix.
- `rotMatToRotVec()`, to obtain the orientation error in vector form, given a rotation matrix.

Implement your solution in `Q4_inverse_dynamics_control.m`. You can test your controller (or inspect the solution) by running `simulate_Q4.m`. If `use_solution` is set to 1, executing this script will show you how the solution should look like. If `use_solution` is set to 0, at each time-step the simulator will use the joint torques returned by your own implementation.

Hint: While an angular velocity reference is not provided to the controller, we would like to limit the rotational motion of the tray during the task. How would you formulate this in terms of angular velocity tracking?

Hint: What can we use to map forces in the task space into the joint space?